

Webserver

Einführung

Was passiert beim Aufruf einer Website?

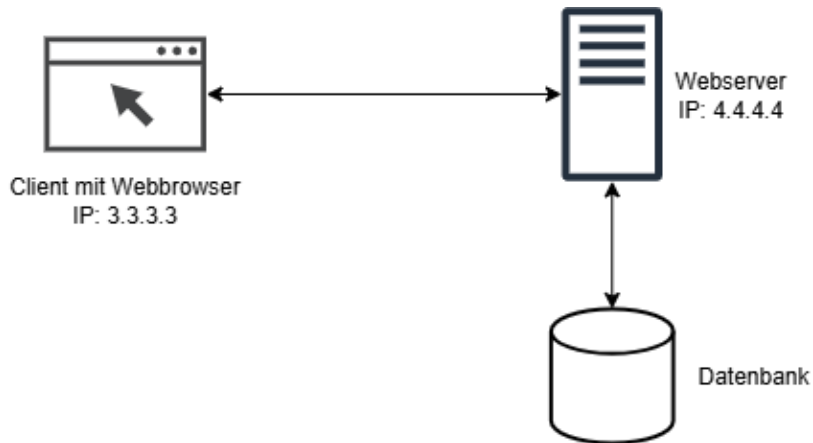


Abbildung 1. Ablauf Webkommunikation

Ein Nutzer möchte die Website `http://www.schule.de/info/klassen.html` aufrufen. Dabei passiert folgendes

1. Der Browser des Clients (IP: 3.3.3.3) baut eine Verbindung zum Webserver (IP: 4.4.4.4) auf. Es wird hierbei eine GET Anfrage an den Webserver gesendet:

```
GET /info/klassen.html
```

TERMINAL

2. Der Webserver empfängt die Anfrage, beschafft bzw. lädt sich aus dem `DocumentRoot` die gewünschte Datei `klassen.html` und sendet diese zurück an den Client.
3. Der Browser des Clients empfängt die Datei und stellt diese dar.

Aus dieser Kommunikation heraus, lässt sich das nachfolgende Datenpaket skizzieren:

Sicherungsschicht	Vlan*	Vermittlungsschicht	Transportschicht	TLS*	Anwendungsschicht	Daten	CRC
00:1A:2B:3C:4D:5E; 52:54:00:12:34:56	...	4.4.4.4; 3.3.3.3	80; >1023	...	www.schule.de; GET /info/klassen.html	...	...

Wesentliche Begrifflichkeiten

URL

URL steht für `Uniform Resource Locator` und ist eine standardisierte Adresse, die den Ort einer Ressource im Internet angibt. Eine URL besteht aus mehreren Teilen und hat immer den nachfolgenden Aufbau: (vgl. [WEB1])

```
protokoll://dnsName/urIPfad
```

TERMINAL

- `protokoll` : Gibt das verwendete Protokoll an, z.B. `http`, `https`, `ftp`, etc.
- `dnsName` : Der Domain-Name oder die IP-Adresse des Servers, auf dem die Ressource gehostet wird, z.B. `www.schule.de` oder `4.4.4.4`.
- `urIPfad` : Der Pfad zur Ressource auf dem Server, z.B. `/info/klassen.html`.

DocumentRoot

Hierbei handelt es sich um das Hauptverzeichnis, aus dem der Webserver die Dateien an den Browser zurückliefert. Sollte als DocumentRoot beispielsweise `/var/www/html` eingetragen sein und eine GET Anfrage nach `http://www.schule.de/info/klassen.html` stattfindet, dann sucht der Webserver unter `/var/www/html/info/klassen.html` . (vgl. [WEB2])

HTTP-Methoden

HTTP-Methoden sind standardisierte Befehle, die von einem Client (z.B. einem Webbrowser) an einen Webserver gesendet werden, um bestimmte Aktionen auf einer Ressource (z.B. einer Webseite oder Datei) durchzuführen. Die wichtigsten HTTP-Methoden sind: (vgl. [WEB3])

- GET : Wird verwendet, um eine Ressource vom Server anzufordern. Dies ist die am häufigsten verwendete Methode und wird typischerweise zum Abrufen von Webseiten verwendet. GET bietet als Möglichkeit, dass auch Parameter innerhalb der URL übergeben werden z. B. `http://www.schule.de/info/klassen.html?name=Heitzer&vorname=Stefan` . Man spricht von sogenannten Query-Parametern. Diese werden mit einem `?` eingeleitet und mit einem `&` getrennt.
- POST : Wird verwendet, um Daten an den Server zu senden, z. B. bei dem Ausfüllen eines Formulars. Die Daten werden im Body der Anfrage gesendet.
- PUT : Wird verwendet, um eine Ressource auf dem Server zu erstellen oder zu aktualisieren. Die Daten werden ebenfalls im Body der Anfrage gesendet.
- DELETE : Wird verwendet, um eine Ressource auf dem Server zu löschen.

Virtual Hosts

Virtual Hosts ermöglichen es, mehrere Websites auf einem einzigen Webserver zu hosten. Dies ist besonders nützlich, wenn man verschiedene Domains oder Subdomains auf demselben Server betreiben möchte. Ein Beispiel hierfür könnte sein: (vgl. [WEB4])

- `www.schule.de` → Website für die Schule. Sucht im DocumentRoot nach `/var/www/schule` .
- `www.firma.de` → Website für eine Firma. Sucht im DocumentRoot nach `/var/www/firma` .

Theorie zu Webservern

Zustandsloses Protokoll HTTP

Das Hypertext Transfer Protocol (HTTP) ist ein zustandsloses Protokoll, welches innerhalb des Application Layers des TCP/IP-Modells arbeitet. Zustandslos in diesem Kontext bedeutet, dass jede Anfrage eines Clients an einen Server für sich alleine steht. Der Server merkt sich somit nicht, was vorher schon passiert ist. (vgl. [WEB5])

Ein einfaches Beispiel

Ein Client möchte einen Webshop besuchen. Hierfür öffnet er den Browser seiner Wahl und gibt dort die URL des Shops ein. Der Browser baut hierbei eine HTTP Verbindung zum Webshop auf. Sobald der Client jetzt anfängt auf dem Webshop die unterschiedlichsten Produkte in den Warenkorb zu legen, merkt sich der Webserver das grundlegend nicht! Jeder Klick auf "In den Warenkorb" wird als einzelne, unabhängige Anfrage betrachtet. Der Webserver weiß somit nicht, welche Produkte der Client schon in den Warenkorb gelegt hat. (vgl. [WEB5])

Das Problem ist daher, dass der Server Dinge wie den Warenkorb, Logins oder Einstellungen nicht erkennen bzw. speichern kann. Für den Webserver sieht jede Anfrage so aus, als wäre es die erste und einzige Anfrage eines bestimmten Clients!

Sessions und Cookies

Um das Problem der Zustandslosigkeit zu umgehen, wurden Mechanismen wie Sessions und Cookies entwickelt. Diese ermöglichen es, Informationen über den Zustand eines Clients zu speichern und wiederzuverwenden. (vgl. [WEB6])

Sessions

Eine Session ist eine serverseitige Speicherung von Informationen über einen bestimmten Client. Wenn ein Client eine Anfrage an den Server sendet, kann der Server eine eindeutige ID für die Session generieren und diese an den Client zurücksenden. Der Client speichert diese ID und sendet sie bei jeder weiteren Anfrage zurück an den Server. Der Server kann dann diese ID verwenden, um den Zustand des Clients zu verfolgen und entsprechende Informationen abzurufen. (vgl. [WEB6])

NOTE

HTTP wird hierdurch nicht zustandsbehaftet! Es wird lediglich ein Mechanismus genutzt, um den Zustand zu simulieren.

Cookies

Cookies sind kleine Textdateien, die vom Server an den Client gesendet und im Browser des Clients gespeichert werden. Diese Cookies können Informationen wie ID von Sessions, Benutzereinstellungen oder andere Daten enthalten. Bei jeder weiteren Anfrage sendet der Browser die gespeicherten Cookies zurück an den Server, wodurch der Server den Zustand des Clients erkennen kann. (vgl. [WEB6])

Allgemeine Übersicht

- HTTP ist ein zustandsloses Protokoll, das im Application Layer des TCP/IP-Modells arbeitet.
- TCP sorgt für eine zuverlässige Übertragung der Daten zwischen Client und Server und ist somit verbindungsorientiert, merkt sich aber nicht den Zustand des Clients.
- UDP ist ein verbindungsloses Protokoll, das für schnelle, aber unzuverlässige Übertragungen verwendet wird.
- IP ist für die Adressierung und das Routing von Datenpaketen im Netzwerk verantwortlich, allerdings ebenfalls verbindungslos. Pakete können schließlich verloren gehen oder falsch ankommen.
- SSH ist ein sicheres, verbindungsorientiertes Protokoll, das für den Fernzugriff auf Systeme verwendet wird.

Nginx - Grundlagen, Installation und Konfiguration

Was ist Nginx?

Nginx ist ein moderner und sehr schneller Webserver, welcher sehr häufig auch - neben seiner Grundfunktionalität als Webserver - als Reverse Proxy oder Load Balancer eingesetzt wird. Zu den wesentlichen Vorteilen von Nginx zählen unter anderem: (vgl. [NGX1])

- Hohe Performance
- Sehr einfache Konfiguration mit Hilfe einzelner Dateien
- Geringer Ressourcenverbrauch
- Modularer Aufbau mit vielen Erweiterungen (Module)
- Gute Zertifikatunterstützung (z.B. Let's Encrypt)
- Perfekt für den Einsatz als Reverse Proxy und Load Balancer geeignet
- Gut geeignet für mehrere virtuelle Hosts auf einem Server

Erste Schritte mit Nginx

Installation

Um Nginx auf einem System zu installieren, muss - abhängig von Linux Distribution - der entsprechende Paketmanager verwendet werden. Unter Debian kann nachfolgender Befehl verwendet werden:

```
apt install nginx
```

TERMINAL

Nach erfolgreicher Ausführung kann der Webserver gestartet und der Status überprüft werden:

NOTE

In der Regel ist der Webserver nach der Installation bereits gestartet!

```
systemctl start nginx
systemctl status nginx
```

TERMINAL

Bei einem Aufruf der IP des Servers im Browser sollte nun die Willkommensseite von Nginx angezeigt werden.

Man will doch nicht immer mit der IP arbeiten, oder?

Um dieses Problem zu umgehen, kann ein entsprechender DNS verwendet werden! Siehe Kapitel 5.0!

Grundlegende Konfiguration

DocumentRoot anpassen

Der DocumentRoot von Nginx ist in der Standardkonfiguration unter `/var/www/html` zu finden. Dort können im Anschluss eigene `.html` abgelegt werden, die der Webserver ausliefern soll.

IMPORTANT

Nach einer Neuinstallation von Nginx kann es sein, dass das Verzeichnis `/var/www/html` als Rechte `root:root` besitzt. In diesem Fall muss das Verzeichnis entsprechend angepasst werden, damit der Webserver darauf zugreifen kann. Nginx nutzt in der Regel den Benutzer `www-data` um auf die Dateien zuzugreifen.

```
chown -R <Your_User>:www-data /var/www
```

TERMINAL

Um wirklich sicherzugehen, dass der entsprechende Benutzer `www-data` ist, kann die Konfigurationsdatei `/etc/nginx/nginx.conf` überprüft werden. Dort sollte sich eine Zeile mit dem nachfolgenden Inhalt befinden:

```
...
user www-data;
...
```

TERMINAL

Arbeiten mit Virtual Hosts

Einführung

Im Standard greift ein Webserver (z.B. Nginx oder Apache) auf ein einziges DocumentRoot zu, um die angeforderten Dateien auszuliefern. Möchte man jedoch mehrere Webseiten auf einem einzigen Webserver hosten, so kommt das Konzept der Virtual Hosts zum Einsatz. Hierbei wird für jede Webseite ein eigener Virtual Host definiert, der auf ein spezifisches DocumentRoot verweist. Dies ermöglicht es, verschiedene Domains oder Subdomains auf demselben Server zu betreiben. (vgl. [WEB4])

`/etc/nginx/sites-available/` und `/etc/nginx/sites-enabled/`

Nginx verwendet zwei Verzeichnisse, um die Konfiguration von Virtual Hosts zu verwalten: `sites-available` und `sites-enabled`.

- `sites-available` : In diesem Verzeichnis werden die Konfigurationsdateien für alle verfügbaren Virtual Hosts gespeichert. Jede Datei repräsentiert einen Virtual Host und enthält die spezifischen Einstellungen, wie z.B. den `server_name`, das `root` Verzeichnis und andere relevante Konfigurationen.
- `sites-enabled` : In diesem Verzeichnis werden symbolische Links zu den Konfigurationsdateien in `sites-available` erstellt, die tatsächlich vom Nginx Webserver verwendet werden. Nur die Virtual Hosts, deren Konfigurationsdateien in `sites-enabled` verlinkt sind, werden vom Webserver aktiv bedient.

```
sites-available
default
```

Nach der Installation von Nginx befindet sich im Verzeichnis `sites-available` bereits eine Standardkonfigurationsdatei mit dem Namen `default`. Diese Datei definiert einen grundlegenden Virtual Host, der auf das `DocumentRoot` `/var/www/html` verweist. Der typische Inhalt dieser Datei sieht wie folgt aus:

TERMINAL

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name _;

    root /var/www/html;
    index index.html index.htm;

    access_log /var/log/nginx/default.access.log;
    error_log /var/log/nginx/default.error.log;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

- `listen 80 default_server;` : Der Virtual Host hört auf Port 80 (HTTP) und ist der Standardserver für eingehende Anfragen.
- `listen [::]:80 default_server;` : Der Virtual Host hört auch auf IPv6 Adressen auf Port 80.
- `server_name...` : Der Platzhalter `_` bedeutet, dass dieser Virtual Host für alle Domainnamen antwortet, die nicht explizit einem anderen Virtual Host zugeordnet sind.
- `root /var/www/html;` : Das `DocumentRoot` für diesen Virtual Host ist `/var/www/html`.
- `index index.html index.htm;` : Die Dateien, die ausgeliefert werden, wenn keine spezifische Datei angefordert wird.
- `access_log` und `error_log` : Pfade zu den Logdateien für Zugriffe und Fehler.
- `location / { ... }` : Definiert das Verhalten für Anfragen an den Wurzelpfad. In diesem Fall wird versucht, die angeforderte Datei nach einer Reihenfolge zu finden, andernfalls wird ein `404` Fehler zurückgegeben:
 - `$uri` : Exakte Datei, also `/index.html`.
 - `$uri/` : Verzeichnis, also `/verzeichnis/index.html`.
 - `=404` : Nichts gefunden, daher HTTP 404 zurückgeben.

Eigenen virtuellen Host erzeugen

TERMINAL

```
server {
    listen 80;
    listen [::]:80;

    server_name <sub>.<name>.lan;

    root /var/www/<sub>.<name>.lan/html;
    index index.html index.htm;

    access_log /var/log/nginx/<sub>.<name>.lan.access.log;
    error_log /var/log/nginx/<sub>.<name>.lan.error.log;

    server_tokens off;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

`sites-enabled`

Um den neu erstellten Virtual Host bzw. auch den `default` zu aktivieren, muss jeweils ein symbolischer Link von der Konfigurationsdatei in `sites-available` zu `sites-enabled` erstellt werden. Dies kann mit dem folgenden Befehl erreicht werden:

```
ln -s /etc/nginx/sites-available/default /etc/nginx/sites-enabled/

ln -s /etc/nginx/sites-available/<sub>.<name>.lan /etc/nginx/sites-enabled/
```

TERMINAL

Nachdem die symbolischen Links erstellt wurden, muss Nginx neu geladen werden, damit die Änderungen wirksam werden:

```
systemctl reload nginx
```

TERMINAL

Testen der Virtual Hosts

Um die Funktionalität der eingerichteten Virtual Hosts zu testen, kann der Webbrowser verwendet werden. Hierzu muss die entsprechende Domain in die Adressleiste des Browsers eingegeben werden. Zum Beispiel:

```
http://<name>.lan

http://<sub>.<name>.lan
```

TERMINAL

IMPORTANT

Die Anfragen funktionieren nur, wenn wirklich im DNS die entsprechenden Einträge auch gepflegt wurden!

Nginx als Reverse Proxy

Was ist ein Reverse Proxy?

Ein Reverse Proxy ist ein Server, der Anfragen von Clients entgegennimmt und diese an einen oder mehrere Services oder gar Server im Backend weiterleitet. Der Reverse Proxy agiert dabei als Vermittler zwischen dem Client und den Services bzw. den Servern. Dies bietet mehrere Vorteile, darunter: (vgl. [WEB7])

- **Sicherheit:** Der Reverse Proxy kann als Schutzschild fungieren, indem er direkte Zugriffe auf die Services bzw. die Server verhindert und Sicherheitsfunktionen bereitstellt. (vgl. [WEB7])
- **Lastverteilung:** Der Reverse Proxy kann Anfragen auf mehrere Services bzw. Server verteilen, um die Last zu balancieren und die Performance zu verbessern. (vgl. [WEB7])
- **Caching:** Der Reverse Proxy kann häufig angeforderte Inhalte zwischenspeichern, um die Antwortzeiten zu verkürzen und die Belastung der Services bzw. der Server zu reduzieren. (vgl. [WEB7])

Erstellung von Backend Services

Um das Prinzip des Reverse Proxys zu veranschaulichen, werden zwei einfache Backend Services mit Python erstellt. Diese Services werden später vom Nginx Reverse Proxy angesprochen.

Auf dem Rechner werden hierbei unter `/var/www/` zwei neue Verzeichnisse `service1` und `service2` erstellt. In beiden Verzeichnissen liegt jeweils eine einfache `index.html` Datei, die den jeweiligen Service identifiziert:

```
<html>
  <head>
    <title>Service 1</title>
  </head>
  <body>
    <h1>Willkommen bei Service 1</h1>
  </body>
</html>
```

TERMINAL

```
<html>
  <head>
    <title>Service 2</title>
  </head>
  <body>
    <h1>Willkommen bei Service 2</h1>
  </body>
</html>
```

Anschließend werden in beiden Verzeichnissen jeweils ein einfacher Python HTTP Server gestartet, der auf unterschiedlichen Ports lauscht:

```
python3 -m http.server 8001 --directory /var/www/service1
python3 -m http.server 8002 --directory /var/www/service2
```

In diesem Beispiel lauscht der erste Service auf Port 8001 und der zweite Service auf Port 8002. Beide Services liefern die jeweiligen `index.html` Dateien aus ihren Verzeichnissen aus.

Einrichtung des Reverse Proxy

Um den Reverse Proxy einzurichten, kann erneut ein neuer Virtual Host in Nginx erstellt werden, der als Reverse Proxy fungiert. Hierzu wird eine neue Konfigurationsdatei in `/etc/nginx/sites-available/` angelegt, z. B. `reverse`.

`<name>.lan`, mit folgendem Inhalt:

```
server {
    listen 80;
    listen [::]:80;

    server_name service1.<name>.lan;

    location / {
        proxy_pass http://127.0.0.1:8001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

server {
    listen 80;
    listen [::]:80;

    server_name service2.<name>.lan;

    location / {
        proxy_pass http://127.0.0.1:8002;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

Interessant sind hier eigentlich nur die Eigenschaften innerhalb der `location / { ... }` Blöcke:

- `proxy_pass http://...;` : Hier wird die Adresse des Backend Services angegeben, an den die Anfragen weitergeleitet werden sollen. In diesem Fall sind es die beiden zuvor erstellten Python HTTP Server, die auf den Ports 8001 und 8002 lauschen.
- `proxy_set_header Host $host;` : Diese Anweisung sorgt dafür, dass der ursprüngliche Header des Hosts der Anfrage an den Backend Service weitergegeben wird. Dies ist wichtig, damit der Backend Service weiß, für welche Domain die Anfrage bestimmt ist.

- `proxy_set_header X-Real-IP $remote_addr;` : Diese Anweisung fügt einen zusätzlichen Header `X-Real-IP` hinzu, der die IP-Adresse des ursprünglichen Clients enthält. Dies kann nützlich sein, wenn der Backend Service die IP-Adresse des Clients benötigt, z. B. für Logging oder Zugriffskontrolle.

Auch hier müssen natürlich wieder die entsprechenden symbolischen Links in `sites-enabled` erstellt und Nginx neu geladen werden, damit die Änderungen wirksam werden:

```
ln -s /etc/nginx/sites-available/reverse.<name>.lan /etc/nginx/sites-enabled/
```

TERMINAL

```
systemctl reload nginx
```

IMPORTANT

Da es zwei neue Domainnamen (`service1.<name>.lan` und `service2.<name>.lan`) gibt, müssen auch im DNS die entsprechenden `A` Einträge erstellt werden, die auf die IP-Adresse des Servers zeigen, auf dem der Nginx Reverse Proxy läuft. Ansonsten können die Domains nicht aufgelöst werden und die Anfragen erreichen den Reverse Proxy nicht! Hierfür kann wieder `nsupdate` verwendet werden.

Filterung von IPs und Authentifizierung

Zugriffsschutz ist natürlich auch für einen Webserver sehr wichtig. Der Webserver stellt Inhalte zur Verfügung, die nicht für jeden einsehbar sein sollten. Nginx selbst bietet dabei mehrere Möglichkeiten für den Zugriffsschutz an.

Basic Authentication

Die wohl einfachste Möglichkeit, den Zugriff auf bestimmte Bereiche einer Webseite zu schützen, ist die sogenannte Basic Authentication. Dabei wird der Benutzer aufgefordert, einen Benutzernamen und ein Passwort einzugeben, bevor er Zugriff auf die geschützten Inhalte erhält. Diese Methode ist einfach zu implementieren, bietet jedoch nur einen grundlegenden Schutz, da die Anmeldedaten unverschlüsselt übertragen werden, sofern kein HTTPS verwendet wird. Hauptanwendungszweck ist für interne Netzwerke und / oder Testumgebungen. (vgl. [WEB9])

Damit die Basic Authentication eingerichtet werden kann, muss zunächst eine Passwortdatei erstellt werden, die die Benutzernamen und Passwörter enthält. Dies kann mit dem Tool `htpasswd`. Sofern das Tool noch nicht vorhanden sein soll, kann dieses über den aufgeführten Befehl hinzugefügt werden:

```
apt update
apt install apache2-utils
```

TERMINAL

Der folgende Befehl erstellt eine neue Passwortdatei und fügt zwei Benutzer hinzu:

```
htpasswd -c /etc/nginx/.htpasswd user1
htpasswd /etc/nginx/.htpasswd user2
```

TERMINAL

Der erste Befehl erstellt die Datei `/etc/nginx/.htpasswd` und fügt den Benutzer `user1` hinzu. Der zweite Befehl fügt den Benutzer `user2` zur bestehenden Datei hinzu. Beim Ausführen der Befehle wird der Benutzer aufgefordert, ein Passwort für jeden Benutzer einzugeben.

Nachdem die Passwortdatei erstellt wurde, muss die Konfigurationsdatei angepasst werden, um die Basic Authentication zu aktivieren. Dazu wird der folgende Block in den entsprechenden `server` oder `location` Sektion der Konfiguration eingefügt. Als Beispiel wird hier die Seite `<sub>.<name>.lan` abgesichert:


```

server {
    listen 80;
    listen [::]:80;

    server_name <sub>.<name>.lan;

    root /var/www/<sub>.<name>.lan/html;
    index index.html index.htm;

    access_log /var/log/nginx/<sub>.<name>.lan.access.log;
    error_log /var/log/nginx/<sub>.<name>.lan.error.log;

    server_tokens off;

    auth_basic "StrengGeheim";
    auth_basic_user_file /etc/nginx/.htpasswd;

    location / {
        try_files $uri $uri/ =404;
    }
}

```

Die Einstellung `auth_basic` legt den Text fest, der im Anmeldefenster angezeigt wird. Die Einstellung `auth_basic_user_file` gibt den Pfad zur Passwortdatei an.

IP-Filterung

Eine weitere Möglichkeit, den Zugriff auf bestimmte Bereiche einer Webseite zu beschränken, ist die IP-Filterung. Dabei wird der Zugriff basierend auf der IP-Adresse des Clients erlaubt oder verweigert. Dies kann nützlich sein, um den Zugriff auf bestimmte IP-Adressen oder IP-Bereiche zu beschränken.

Um die IP-Filterung einzurichten, wird der folgende Block in den entsprechenden `server` oder `location` Abschnitt der Konfiguration eingefügt. Als Beispiel wird hier der Zugriff auf die Seite `<sub>.<name>.lan` nur für eine entsprechende IP erlaubt:

```

server {
    listen 80;
    listen [::]:80;

    server_name <sub>.<name>.lan;

    root /var/www/<sub>.<name>.lan/html;
    index index.html index.htm;

    access_log /var/log/nginx/<sub>.<name>.lan.access.log;
    error_log /var/log/nginx/<sub>.<name>.lan.error.log;

    server_tokens off;

    allow <IP>;
    deny all;

    location / {
        try_files $uri $uri/ =404;
    }
}

```

Wichtig hierbei ist, dass es mehrere `allow` Einträge geben kann und auch komplette Netze angegeben werden können!

Umstellung auf HTTPS

Um die Sicherheit einer Webseite zu erhöhen, ist es ratsam, den Datenverkehr zwischen dem Webserver und den Clients zu verschlüsseln. Dies wird durch die Verwendung von HTTPS (Hypertext Transfer Protocol Secure) erreicht, das SSL/TLS-Zertifikate nutzt, um die Kommunikation zu sichern. (vgl. [WEB8])

Begriffserklärungen

TLS/SSL

TLS (Transport Layer Security) und sein Vorgänger SSL (Secure Sockets Layer) sind kryptographische Protokolle, die sichere Kommunikation über ein Computernetzwerk ermöglichen. Sie werden häufig in Webbrowsern und anderen Anwendungen verwendet, die eine sichere Datenübertragung erfordern. TLS/SSL bieten Verschlüsselung, Datenintegrität und Authentifizierung, um sicherzustellen, dass die übertragenen Daten vor Abhören, Manipulation und Fälschung geschützt sind. (vgl. [WEB8])

Asymmetrische Public-Key-Verschlüsselung

Die asymmetrische Public-Key-Verschlüsselung ist ein kryptographisches Verfahren, bei dem ein Paar von Schlüsseln verwendet wird: (vgl. [WEB8])

- **Private Key:** Dieser Schlüssel wird geheim gehalten und ist nur dem Besitzer bekannt. Er wird verwendet, um Daten zu entschlüsseln, die mit dem zugehörigen öffentlichen Schlüssel verschlüsselt wurden, oder um digitale Signaturen zu erstellen. Der Schlüssel befindet sich folglich beim Besitzer einer Website auf dem Webserver. (vgl. [WEB8])
- **Public Key:** Dieser Schlüssel wird öffentlich zugänglich gemacht und kann von jedem verwendet werden, um Daten zu verschlüsseln, die nur mit dem zugehörigen privaten Schlüssel entschlüsselt werden können, oder um die Echtheit digitaler Signaturen zu überprüfen. Der öffentliche Schlüssel wird in einem SSL/TLS-Zertifikat veröffentlicht, das von einer vertrauenswürdigen Zertifizierungsstelle (CA) ausgestellt wird. (vgl. [WEB8])

Certificate Authority (CA)

Eine Certificate Authority (CA) ist eine vertrauenswürdige Organisation oder ein Unternehmen, das digitale Zertifikate ausstellt und verwaltet. Diese Zertifikate werden verwendet, um die Identität von Websites, Einzelpersonen oder Organisationen zu bestätigen und eine sichere Kommunikation über das Internet zu ermöglichen. Eine CA überprüft die Identität des Antragstellers, bevor sie ein Zertifikat ausstellt, und stellt sicher, dass die ausgestellten Zertifikate den Sicherheitsstandards entsprechen. Bekannte CAs sind beispielsweise Let's Encrypt, DigiCert und GlobalSign. (vgl. [TSL1])

Eine CA kann dabei auch selbst betrieben werden, z. B. in einem Unternehmensnetzwerk, um interne Zertifikate auszustellen und zu verwalten. In diesem Fall muss jedoch sichergestellt werden, dass alle Clients im Netzwerk der internen CA vertrauen, damit die ausgestellten Zertifikate als gültig anerkannt werden. (vgl. [TSL1])

Aufsetzen einer eigenen CA

Anlage einer entsprechenden Ordnerstruktur

Die erzeugten Zertifikate und Schlüssel sollten in einer gut organisierten Ordnerstruktur abgelegt werden, um die Verwaltung zu erleichtern. Eine mögliche Struktur könnte wie folgt aussehen:

```
mkdir -p /root/ca/certs
mkdir -p /root/ca/private
mkdir -p /root/ca/csrs
```

TERMINAL

`certs` enthält die ausgestellten, öffentlichen Zertifikate, während `private` die privaten Schlüssel der CA speichert. Innerhalb des Ordners `csrs` befinden sich entsprechende Zertifikatsanfragen. Es ist wichtig, dass der `private` Ordner gut geschützt ist, da der private Schlüssel der CA äußerst sensibel ist. Die Zugriffsrechte sollten entsprechend restriktiv gesetzt werden:

```
chmod 600 /root/ca/private
```

NOTE

Alle folgenden Kommandos werden im Ordner `/root/ca` ausgeführt!

Erstellung eines Private Key für die CA

IMPORTANT

Hierbei handelt es sich um den wichtigsten Schlüssel des Systems!

Der erste Schritt beim Aufsetzen einer eigenen CA ist die Erstellung eines privaten Schlüssels, der zur Signierung der Zertifikate verwendet wird. Dies kann mit dem folgenden `openssl` Befehl durchgeführt werden:

```
openssl genrsa -out private/ca.key 4096
```

TERMINAL

Dieser Befehl generiert einen RSA-Schlüssel (= `genrsa`) mit einer Länge von 4096 Bit und speichert ihn in der Datei `private/ca.key`. Als Tool wird hierbei `openssl` verwendet, das eine weit verbreitete Open-Source-Software für die Implementierung von SSL und TLS ist.

IMPORTANT

Dieser Schlüssel muss sicher aufbewahrt werden, da er zur Signierung der Zertifikate verwendet wird und somit die Vertrauenswürdigkeit der CA bestimmt! Wer ihn hat bzw. Zugriff darauf bekommt, kann Zertifikate fälschen!

Erstellung eines selbstsignierten CA-Zertifikats

Nachdem der private Schlüssel erstellt wurde, kann nun ein selbstsigniertes CA-Zertifikat generiert werden. Dieses Zertifikat wird verwendet, um die Identität der CA zu bestätigen und die ausgestellten Zertifikate zu signieren. Der folgende Befehl erstellt ein selbstsigniertes Zertifikat, das für 3650 Tage (ca. 10 Jahre) gültig ist:

```
openssl req -x509 -new -nodes -key private/ca.key -sha256 -days 3650 -out certs/ca.crt
```

TERMINAL

- `req` : Dieser Befehlsteil gibt an, dass ein Zertifikatsantrag erstellt wird.
- `-x509` : Die Option steht dafür, dass ein selbstsigniertes Zertifikat benötigt wird. Selbstsigniert bedeutet, dass die CA ihr eigenes Zertifikat signiert, anstatt es von einer anderen CA signieren zu lassen.
- `-new` : Hierrüber wird spezifiziert, dass ein komplett neuer Zertifikatsantrag erstellt wird.
- `-nodes` : Diese Option gibt an, dass der private Schlüssel nicht mit einer Passphrase geschützt werden soll. Dies ist besonders wichtig für CA-Schlüssel, da sie häufig automatisiert verwendet werden.
- `-key private/ca.key` : Hier wird der Pfad zum zuvor erstellten privaten Schlüssel der CA angegeben.
- `-sha256` : Diese Option gibt den Hash-Algorithmus an, der zur Signierung des Zertifikats verwendet wird. SHA-256 ist ein sicherer Algorithmus, der weit verbreitet ist.
- `-days 3650` : Hierrüber wird die Gültigkeitsdauer des Zertifikats in Tagen angegeben. 3650 Tage entsprechen etwa 10 Jahren.
- `-out certs/ca.crt` : Der Pfad, unter dem das erzeugte CA-Zertifikat gespeichert werden soll.

Beim Ausführen des Befehls wird der Benutzer aufgefordert, verschiedene Informationen einzugeben, die im Zertifikat enthalten sein sollen, wie z.B. Land, Bundesland, Ort, Organisation, Organisationseinheit und Common Name (CN). Diese Informationen helfen dabei, die Identität der CA zu bestätigen. Als Beispiele können die nachfolgenden Eingaben verwendet werden:

```
Country Name (2 letter code) [AU]: DE
State or Province Name (full name) [Some-State]: Bavaria
Locality Name (eg, city) []: Plattling
Organization Name (eg, company) [Internet Widgits Pty Ltd]: EDV-Schulen Plattling
Organizational Unit Name (eg, section) []: IT
Common Name (e.g. server FQDN or YOUR name) []: <name> Root CA
```

Nachdem diese Informationen erfasst wurden, wird das selbstsignierte CA-Zertifikat in der Datei `certs/ca.crt` gespeichert. Dieses Zertifikat kann nun verwendet werden, um andere Zertifikate zu signieren und somit eine Vertrauenskette aufzubauen.

SAN-Dateien für Zertifikate

SAN ist die Abkürzung für Subject Alternative Name. SAN-Dateien werden verwendet, um zusätzliche Domain-Namen und IP-Adressen in einem SSL/TLS-Zertifikat zu definieren. Dies ist besonders nützlich, wenn ein Zertifikat für mehrere Domains oder Subdomains verwendet werden soll. (vgl. [SAN1])

Erstellung einer SAN-Datei

Um eine SAN-Datei zu erstellen, kann eine einfache Textdatei verwendet werden. Diese Datei sollte die folgenden Einträge enthalten:

```
nano san-web-<name>-lan.cnf
```

In der Datei können nun die einzelnen Domains hinterlegt werden, welche mit einem Zertifikat abgesichert werden sollen:

```
[ req ]
default_bits      = 2048
prompt            = no
default_md        = sha256
req_extensions    = req_ext
distinguished_name = dn

[ dn ]
C = DE
ST = Bavaria
L = Plattling
O = EDV-Schulen Plattling
OU = IT
CN = <name>.lan

[ req_ext ]
subjectAltName = @alt_names

[ alt_names ]
DNS.1 = <name>.lan
DNS.2 = nginx.<name>.lan
DNS.3 = service1.<name>.lan
DNS.4 = service2.<name>.lan
```

Hierbei werden die verschiedenen Domains unter dem Abschnitt `[ alt_names ]` mit dem Präfix `DNS.` aufgelistet. Jede Domain erhält eine fortlaufende Nummerierung (z.B. `DNS.1`, `DNS.2`, etc.). Die ganzen Einträge werden dann unter dem Schlüssel `subjectAltName` zusammengefasst.

Erstellung eigener Zertifikate

Um nun eigene Zertifikate für eine Website zu erstellen, muss in einem ersten Schritt ein Schlüsselpaar generiert werden.

Erzeugung eines privaten Schlüssels

Der private Schlüssel kann mit dem folgenden Befehl erstellt werden:

```
openssl genrsa -out private/web.<name>.lan.key 2048
```

Dieser Befehl generiert einen RSA-Schlüssel mit einer Länge von 2048 Bit und speichert ihn in der Datei `private/web.<name>.lan.key`.

IMPORTANT

Auch hier gilt wieder, dass der private Schlüssel sicher aufbewahrt werden sollte, da er zur Entschlüsselung der Daten verwendet wird, die mit dem zugehörigen öffentlichen Schlüssel verschlüsselt wurden.

Erstellung eines Zertifikatsantrags (CSR)

Eine Zertifikatsanfrage (CSR = Certificate Signing Request) wird benötigt, um ein Zertifikat von einer CA signieren zu lassen. Der folgende Befehl erstellt eine CSR basierend auf dem zuvor generierten privaten Schlüssel:

```
openssl req -new -key private/web.<name>.lan.key -out csrs/web.<name>.lan.csr -config san-web-<name>-lan.cnf
```

Beim Ausführen des Befehls wird der Benutzer aufgefordert, verschiedene Informationen einzugeben, die im Zertifikat enthalten sein sollen, wie z.B. Land, Bundesland, Ort, Organisation, Organisationseinheit und Common Name (CN). Diese Informationen helfen dabei, die Identität der Website zu bestätigen. Hier kann man sich wieder an die bereits erfassten Einträge der CA orientieren!

NOTE

Die Informationen werden nur abgefragt, wenn innerhalb der SAN-Datei kein Abschnitt `[ dn ]` mit den entsprechenden Einträgen vorhanden ist. In diesem Fall werden die Informationen direkt aus der SAN-Datei übernommen und die Abfrage übersprungen.

Zertifikat signieren

In einem letzten Schritt muss die zuvor erstellte Zertifikatsanfrage (CSR) von der eigenen Zertifizierungsstelle (CA) signiert werden, um ein gültiges Zertifikat zu erhalten. Hierfür erstellt man zuerst eine Extension für das Serverzertifikat:

```
nano web-server-ext.cnf
```

Der Inhalt der Datei sieht wie folgt aus:

```
basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = <name>.lan
DNS.2 = nginx.<name>.lan
DNS.3 = service1.<name>.lan
DNS.4 = service2.<name>.lan
```

Im Anschluss kann das Zertifikat mit folgendem Befehl signiert werden:

```
openssl x509 -req \
-in csrs/web.<name>.lan.csr \
-CA certs/ca.crt \
-CAkey private/ca.key \
-CAcreateserial \
-out certs/web.<name>.lan.crt \
-days 825 \
-sha256 \
-extfile web-server-ext.cnf
```

Dieser Befehl signiert die CSR-Datei `csrs/web.<name>.lan.csr` mit dem privaten Schlüssel der CA (`private/ca.key`) und dem Zertifikat der CA (`certs/ca.crt`). Das resultierende signierte Zertifikat wird in der Datei `certs/web.<name>.lan.crt` gespeichert und ist für 825 Tage gültig. Die Option `-extfile web-server-ext.cnf` stellt sicher, dass die in der SAN-Datei definierten alternativen Namen im Zertifikat enthalten sind. `-CAcreateserial` erstellt eine Seriennummer-Datei für die CA, falls diese noch nicht existiert.

Einbindung der Zertifikate

Um die zuvor erstellten und signierten Zertifikate in einem Webserver wie Nginx zu verwenden, müssen diese in der Konfigurationsdatei des Servers eingebunden werden. Dies geschieht in der Regel durch Angabe der Pfade zu den Zertifikatsdateien und den zugehörigen privaten Schlüsseln. Hierfür empfiehlt es sich, die folgenden Schritte zu befolgen:

```
mkdir -p /etc/nginx/ssl
```

In das neu angelegte Verzeichnis `/etc/nginx/ssl` werden nun die signierten Zertifikate und die privaten Schlüssel kopiert:

```
cp certs/web.<name>.lan.crt /etc/nginx/ssl/
cp private/web.<name>.lan.key /etc/nginx/ssl/
```

Im Anschluss kann jeder einzelne virtuelle Hosts konfiguriert werden, damit HTTPS auch unterstützt wird. Hierzu wird die Konfigurationsdatei des jeweiligen Virtual Hosts in `/etc/nginx/sites-available/` angepasst, z. B. `default.<name>.lan`:

```
server {
    listen 443 ssl default_server;
    listen [::]:443 ssl default_server;

    server_name _;

    ssl_certificate /etc/nginx/ssl/web.<name>.lan.crt;
    ssl_certificate_key /etc/nginx/ssl/web.<name>.lan.key;

    root /var/www/html;
    index index.html index.htm;

    access_log /var/log/nginx/default.access.log;
    error_log /var/log/nginx/default.error.log;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Wichtig ist weiterhin, dass der Webserver so konfiguriert wird, dass er sowohl auf Port 80 (HTTP) als auch auf Port 443 (HTTPS) lauscht. Hierfür kann ein zusätzlicher Server-Block für HTTP hinzugefügt werden, der alle Anfragen auf HTTPS umleitet. Die entsprechende Datei kann dabei den Namen `http-redirect` tragen und nachfolgenden Inhalt haben:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name
        <name>.lan
        nginx.<name>.lan
        service1.<name>.lan
        service2.<name>.lan;

    return 301 https://$host$request_uri;
}
```

Auch hier gilt wieder, dass nach der Anpassung der Konfigurationsdateien die entsprechenden symbolischen Links in `sites-enabled` erstellt und Nginx neu geladen werden muss, damit die Änderungen wirksam werden:

```
ln -s /etc/nginx/sites-available/http-redirect /etc/nginx/sites-enabled/
```

```
systemctl reload nginx
```

IMPORTANT

Mit dem Kommando `nginx -t` kann vor dem Neustart von Nginx überprüft werden, ob die Konfigurationsdateien fehlerfrei sind. Dies ist besonders wichtig, um sicherzustellen, dass der Webserver nach den Änderungen weiterhin korrekt funktioniert und keine Syntaxfehler in den Konfigurationsdateien vorhanden sind.

Finale Schritte

Das Zertifikat muss dem System im Anschluss noch bekannt gemacht werden, damit es auch von Webservern etc. verwendet werden kann. Hierzu muss das Root-CA Zertifikat in das Verzeichnis `/usr/local/share/ca-certificates/` kopieren und anschließend mit dem Befehl `update-ca-certificates` die Zertifikate aktualisieren:

```
cp certs/ca.crt /usr/local/share/ca-certificates/ca.crt
update-ca-certificates
```

Dadurch wird das Root-CA Zertifikat in den Systemweiten Zertifikatsspeicher aufgenommen und steht somit allen Anwendungen zur Verfügung, die auf diesen zugreifen, wie z. B. Webbrowser oder andere Clients, die SSL/TLS-Verbindungen herstellen.

IMPORTANT

Damit Webbrowser wie Firefox oder Chrome dem selbstsignierten Zertifikat vertrauen, muss das Root-CA Zertifikat in den jeweiligen Zertifikatsspeicher der Browser importiert werden. Hierfür sind folgende Schritte notwendig:

Auslieferung des Root-CA Zertifikats an Clients

Um sicherzustellen, dass alle Clients dem selbstsignierten Zertifikat vertrauen, muss das Root-CA Zertifikat auf alle Clients verteilt und dort installiert werden:

```
scp /root/ca/certs/ca.crt <User>@<IP_of_Client>:/tmp/
```

Auf dem Client muss das Zertifikat im Anschluss in den Zertifikatsspeicher des Betriebssystems importiert werden:


```
cp /tmp/ca.crt /usr/local/share/ca-certificates/ca.crt
```

```
update-ca-certificates
```

Auch hier gilt wieder, dass für Webbrowser wie Firefox oder Chrome das Root-CA Zertifikat in den jeweiligen Zertifikatsspeicher der Browser importiert werden muss, damit dem selbstsignierten Zertifikat vertraut wird.

Nginx und HTTPS auf einem extra Host

Natürlich kann es vorkommen, dass man einen zweiten Webserver auf einem separaten Host betreiben möchte, der ebenfalls über HTTPS erreichbar sein soll. In diesem Fall müssen die SSL/TLS-Zertifikate entsprechend auf dem neuen Host eingerichtet und konfiguriert werden.

Installation Webserver

Zunächst muss auf dem neuen Host Nginx installiert werden. Dies kann wie gewohnt über den Paketmanager erfolgen:

```
apt update
apt install nginx
```

Konfiguration des Hosts

Innerhalb der `sites-available` gibt es bereits eine Standardkonfigurationsdatei namens `default`, die nur etwas aufgeräumt werden kann bzw. darf:

```
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name _;

    root /var/www/html;
    index index.html index.htm;

    access_log /var/log/nginx/default.access.log;
    error_log /var/log/nginx/default.error.log;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Konfiguration des Eintrags für DNS

Wichtig ist natürlich, dass der neue Host auch über DNS aufgelöst werden kann. Hierfür muss der entsprechende Schlüssel auf den Client kopiert werden, damit der DNS via `nsupdate` auch aktualisiert werden kann:

```
scp /etc/bind/<Name_of_Key>.key <User>@<IP_of_new_Webserver>:/tmp/
```

Anschließend kann der DNS-Eintrag für den neuen Webserver hinzugefügt werden:

```
nsupdate -k /tmp/<Name_of_Key>.key
```

Privaten Schlüssel für den neuen Webserver erstellen

Auf dem neuen Webserver muss nun ein privater Schlüssel für die Zertifikatsanforderung erstellt werden:

```
mkdir -p /etc/nginx/ssl
cd /etc/nginx/ssl

openssl genrsa -out web2.<name>.lan.key 2048
```

Minimale SAN für die Zertifikationsanforderung erstellen

Um ein SAN-Zertifikat zu erstellen, muss eine Konfigurationsdatei für die Zertifikationsanforderung erstellt werden. Diese Datei definiert die zusätzlichen Domainnamen, die im Zertifikat enthalten sein sollen:

```
nano web2-<name>-lan.cnf
```

Der Inhalt der Datei sieht folgendermaßen aus:

```
[ req ]
default_bits      = 2048
prompt           = no
default_md        = sha256
req_extensions    = req_ext
distinguished_name = dn

[ dn ]
C = DE
ST = Bavaria
L = Plattling
O = EDV-Schulen Plattling
OU = IT
CN = web2.<name>.lan

[ req_ext ]
subjectAltName = @alt_names

[ alt_names ]
DNS.1 = web2.<name>.lan
```

Zertifikatsanforderung erstellen

Mit der zuvor erstellten SAN-Datei kann nun die Zertifikatsanforderung (CSR) erstellt werden:

```
openssl req -new \
-key web2.<name>.lan.key \
-out web2.<name>.lan.csr \
-config web2-<name>-lan.cnf
```

Zertifikat von der CA signieren lassen

Die erstellte Zertifikatsanforderung muss nun von der eigenen Zertifizierungsstelle (CA) signiert werden. Hierfür kann die CSR-Datei auf den CA-Server kopiert und dort signiert werden:

```
scp web2.<name>.lan.csr <User>@<IP_of_CA_Server>:/tmp/
```

Auf dem Server der Root-CA kann im Anschluss die Zertifikatsanforderung signiert werden. Wichtig hierbei ist, dass eine Extensionsdatei für das Serverzertifikat erstellt wird. Hierfür wird eine Datei namens `web2-<name>-ext.cnf` mit folgendem Inhalt erstellt:

```

basicConstraints = CA:FALSE
keyUsage = digitalSignature, keyEncipherment
extendedKeyUsage = serverAuth
subjectAltName = @alt_names

[alt_names]
DNS.1 = web2.<name>.lan

```

Um die Zertifikatsanforderung zu signieren, werden folgende Befehle ausgeführt:

```

cp web2.<name>.lan.csr /root/ca/csrs/

cd /root/ca

openssl x509 -req \
  -in csrs/web2.<name>.lan.csr \
  -CA certs/ca.crt \
  -CAkey private/ca.key \
  -CAcreateserial \
  -out certs/web2.<name>.lan.crt \
  -days 825 \
  -sha256 \
  -extfile web2-<name>-ext.cnf

```

Anschließend kann das signierte Zertifikat wieder auf den neuen Webserver kopiert werden:

```
scp /root/ca/certs/web2.<name>.lan.crt <User>@<IP_of_new_Webserver>:/tmp/
```

Auf dem neuen Webserver muss das Zertifikat in das SSL-Verzeichnis verschoben werden:

```
mv /tmp/web2.<name>.lan.crt /etc/nginx/ssl/
```

Anpassung von default für HTTPS

Nun kann die Konfigurationsdatei des virtuellen Hosts angepasst werden, damit HTTPS unterstützt wird. Hierzu wird die Datei default in /etc/nginx/sites-available/ bearbeitet:

```

server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name web2.<name>.lan;

    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl default_server;
    listen [::]:443 ssl default_server;

    server_name web2.<name>.lan;

    ssl_certificate      /etc/nginx/ssl/web2.<name>.lan.crt;
    ssl_certificate_key  /etc/nginx/ssl/web2.<name>.lan.key;

    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}

```

Der Webserver kann im Anschluss neu geladen werden, damit die Änderungen wirksam werden:

```
nginx -t
```

```
systemctl reload nginx
```

TERMINAL